

IOT PROJECT

Line Following Robot

Malik Bogonluri

1. BESCHRIJVING.....	3
2. HARDWARE (HET INITIËLE ONTWERP).....	3
2.1 Het Voedingcircuit (Power Subsystem).....	3
ontrolecentrum (ESP32-C6-mini Subsystem).....	4
2.3 De Sensoren en Motor Drivers (Sensors & Actuators Subsystem).....	5
2.4 De Gebruikersinterface en Connectiviteit (User-Interface Subsystem).....	5
3. SCHEMA & CIRCUIT ONTWERP.....	7
3.1 Pagina 1: Voedingcircuit (Power Subsystem).....	7
3.2 Pagina 2: Centraal Controlecentrum (ESP32-C6-mini).....	8
3.3 Pagina 3: Sensoraansturing & Motor Drivers.....	9
3.4 Pagina 4: Gebruikersinterface & Connectiviteit.....	10
3.5 Architectuur van het Schema.....	10
3.6 Componentenlijst (BOM - Bill of Materials).....	11
4. Het DASHBOARD (GRAFANA VISUALISATIE).....	12
4.1 Batterijstatus (Fuel Gauge).....	12
4.2 Omgevingstemperatuur (BME280 Telemetrie).....	13
4.3 Externe Besturing (Blynk Remote Switch).....	13
5. PROGRAMMACODE.....	14
6. DATASHEET.....	21

1. BESCHRIJVING

Een Line Following Robot is een autonoom voertuig dat ontworpen is om zelfstandig een vooraf gedefinieerd pad te detecteren en te volgen. Meestal bestaat dit pad uit een duidelijke zwarte lijn op een witte ondergrond (of omgekeerd). De robot corrigeert zijn koers continu zonder dat er menselijke besturing of een afstandsbediening aan te pas komt.

2. HARDWARE (HET INITIËLE ONTWERP)

De robot is gebouwd op een op maat ontworpen PCB (Printed Circuit Board). Er is bewust gekozen om géén breadboard te gebruiken. Omdat de robot beweegt, zouden losse draden snel losraken of kortsluiting veroorzaken bij een botsing. Een PCB zorgt voor stevige, gesoldeerde verbindingen.

Het voedingscircuit (Power Subsystem) is opgebouwd uit vier duidelijke onderdelen:

2.1 Het Voedingscircuit (Power Subsystem)

- USB Type-C & Oplader (TP5311): De robot heeft een moderne USB-C aansluiting om de batterij op te laden. De TP5311 chip regelt dit laadproces veilig. Twee LED-lampjes (STDBY en CHRG) laten zien of de batterij nog laadt of al vol is. Een ingebouwde temperatuursensor (NTC) zorgt dat de batterij niet oververhit raakt tijdens het laden.
- Batterijbeveiliging / BMS (HY2120): Om de lithium-ion batterij te beschermen, zit er een BMS-circuit op de printplaat. De HY2120 chip werkt samen met twee MOSFETs (A04406A) om de batterij continu te bewaken. Het systeem grijpt direct in bij overladen, te diep ontladen of kortsluiting.

- Spanningsregelaars (5V Buck & 3.3V LDO): De robot heeft verschillende spanningen nodig om correct en efficiënt te werken:
 - 5V Lijn (TPS562200 Buck): Deze efficiënte omvormer zet de batterijspanning om naar een stabiele 5V. Deze spanning wordt specifiek gebruikt voor de grote matrix van WS2812B RGB-status-LED's en eventuele servo's. Dit voorkomt dat de spanningsval van de LED's de microcontroller stoort.
 - 3.3V Lijn (AMS1117): Deze regelaar maakt van de stabiele 5V een heel schone 3.3V. Dit is noodzakelijk voor de gevoelige ESP32-C6 microcontroller en de sensoren (zoals de BME280).
 - Directe Batterijspanning (+BATT): De DRV8833 motor drivers zijn rechtstreeks aangesloten op de ongereguleerde batterijspanning. Hierdoor krijgen de DC-motoren het maximale vermogen en de maximale stroom van de accu, zonder de 5V of 3.3V regelaars te zwaar te belasten.
- Indicatoren & Batterijmeter (Fuel Gauge): Op de printplaat zitten twee simpele LED-lampjes die direct laten zien of de 5V en 3.3V spanningen werken. Daarnaast zit er een 'Fuel Gauge' (spanningsdeler) op. Hiermee kan de microcontroller zelf meten hoe vol de batterij nog is, zodat de robot weet wanneer hij moet stoppen.

ontrolecentrum (ESP32-C6-mini Subsystem)

Dit deelsysteem vormt het "brein" van de robot. Het verwerkt de gegevens van de sensoren en stuurt de motoren aan. Dit circuit bestaat uit twee hoofdonderdelen:

- De Microcontroller (ESP32-C6-MINI-1): Dit is de centrale processor op de printplaat. De ESP32-C6 is een moderne, krachtige 32-bit controller. Hij leest continu de waarden van de lijnrecalculatie-sensoren in en berekent razendsnel hoe de motoren moeten bijsturen. Daarnaast beschikt deze chip over ingebouwde Wi-Fi 6 en Bluetooth, wat in de toekomst handig is voor draadloze updates of data-analyse. De chip is voorzien van een eigen externe kristaloscillator (Y1) voor een uiterst stabiele kloksnelheid.
- USB-naar-Seriële Programmering (CH343F): Om code (firmware) vanaf de computer naar de ESP32-C6 te kunnen flashen, is er een CH343F chip op de PCB geplaatst. Deze chip vertaalt de USB-signalen van de computer naar seriële data (TXD en RXD) die de microcontroller begrijpt.
 - Twee status-LED's (Yellow en Green) knipperen wanneer er data wordt verzonden of ontvangen, wat erg handig is bij het debuggen.
 - Het circuit regelt ook automatisch de RTS en DTS signalen (via de ER en BOOT pinnen), zodat de microcontroller automatisch in "download-modus" springt wanneer je nieuwe code uploadt vanuit de Arduino IDE.

2.3 De Sensoren en Motor Drivers (Sensors & Actuators Subsystem)

Dit deelsysteem zorgt ervoor dat de robot de lijn kan zien én daadwerkelijk kan rijden. Het circuit bestaat uit drie belangrijke blokken:

- De Motor Drivers (2x DRV8833PW): Om de DC-motoren aan te sturen zijn er twee geavanceerde DRV8833 dual H-brug chips aanwezig (één voor de voorste motoren/wielen F_ en één voor de achterste B_). Omdat de microcontroller zelf te weinig stroom levert, vertalen deze drivers de PWM-signalen van de ESP32-C6 naar krachtige stromen rechtstreeks vanuit de batterij (+BATT). De chips hebben ingebouwde beveiliging tegen oververhitting en kortsluiting. Met de SLEEP pin kan de microcontroller de motoren volledig uitschakelen om stroom te besparen.
- 5-Way Tracing (74HC165 Schuifregister): De robot gebruikt 5 infrarood-lijnsensoren (S1 tot en met S5) om de zwarte lijn te detecteren. In plaats van 5 kostbare pinnen op de ESP32-C6 te gebruiken, worden de sensor-inputs aangesloten op een 74HC165 parallel-in/serial-out schuifregister. Deze chip verzamelt de 5 digitale statussen en stuurt ze via één enkele seriële datalijn (DATA) naar de microcontroller. Dit bespaart veel GPIO-pinnen op het hoofdboard.
- Omgevingssensor (BME280): Als extra uitbreiding is er een BME280 sensor aanwezig. Deze communiceert via I2C (SCL en SDA) met de microcontroller en meet de exacte temperatuur, luchtvochtigheid en luchtdruk. Hoewel dit niet direct nodig is voor het lijnvolgen zelf, kan de robot hiermee tijdens het rijden telemetrie-data over zijn omgeving verzamelen.
- Test Points: Op de PCB zijn vier fysieke testpunten (TP1 tot TP4) geplaatst. Hierdoor kan er tijdens het testen gemakkelijk een oscilloscoop of multimeter worden aangesloten op de kritieke klok- en datalijnen (PL, CP, DATA) om eventuele fouten in de sensorcommunicatie snel op te sporen.

2.4 De Gebruikersinterface en Connectiviteit (User-Interface Subsystem)

Dit laatste deelsysteem zorgt ervoor dat de operator met de robot kan communiceren en dat de status van de robot visueel duidelijk zichtbaar is. Het circuit bestaat uit de volgende onderdelen:

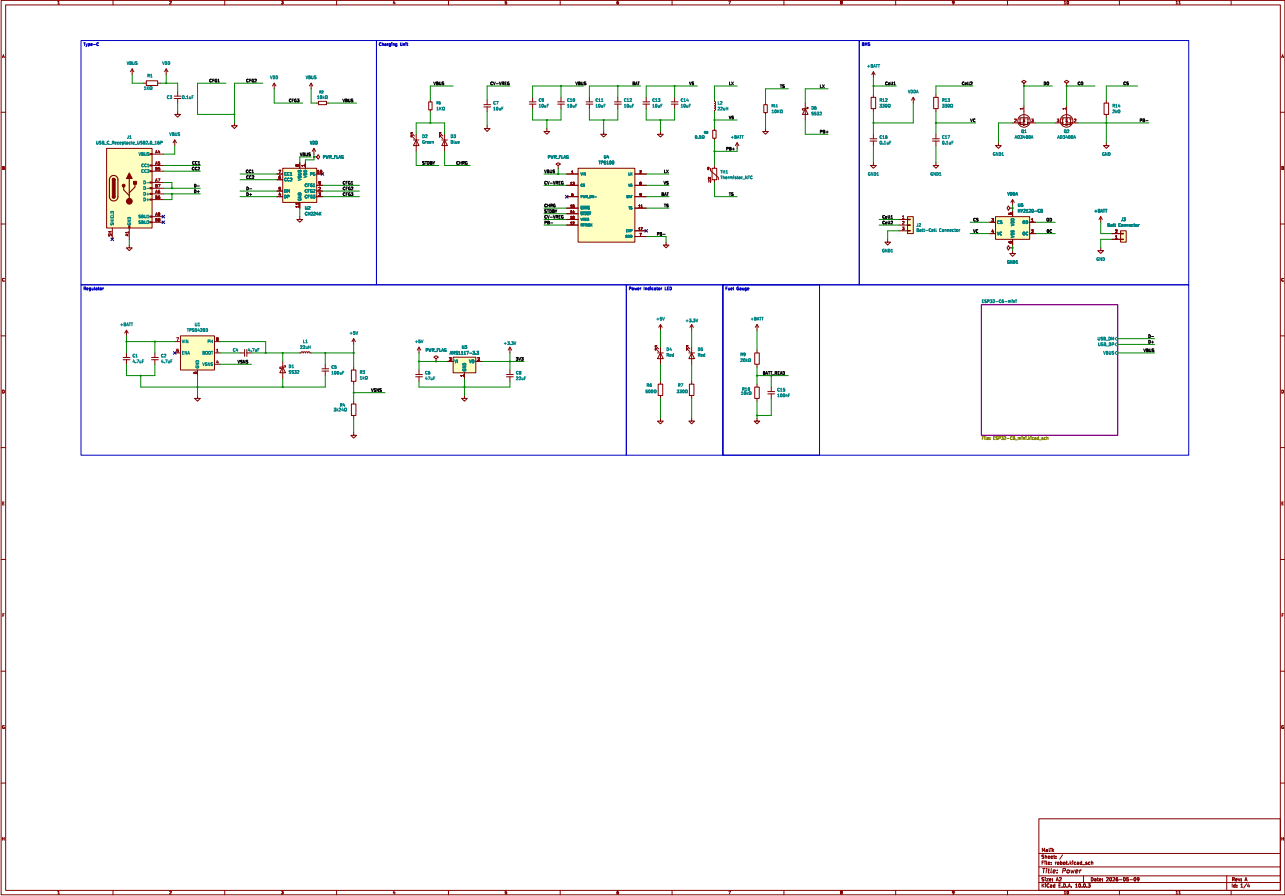
- De RGB Status-LED's (WS2812B / SK6812 NeoPixels): De rechterkant van het schema bevat een lange, in serie geschakelde reeks van adresseerbare RGB-LED's (verdeeld in groepen ULED1 tot ULED4). Omdat ze individueel via één enkele datalijn (PIN_ULED) worden aangestuurd, kan de robot ingewikkelde lichtpatronen, kleuren en animaties tonen. Dit wordt gebruikt om bijvoorbeeld de rijrichting, batterijstatus of foutcodes (zoals "Line Lost") direct op de behuizing te visualiseren.
- LED Voedingsschakelaar (Activation Status LED): Omdat een grote hoeveelheid RGB-LED's constant stroom verbruikt (zelfs als ze uitstaan), bevat de PCB een slimme stroomschakelaar. Via een N-channel MOSFET (A03416A) kan de microcontroller de volledige stroomtoevoer

naar de LED-strips via de `ULED_GND` lijn softwarematig in- of uitschakelen. Dit helpt enorm om energie te besparen wanneer de robot stilstaat.

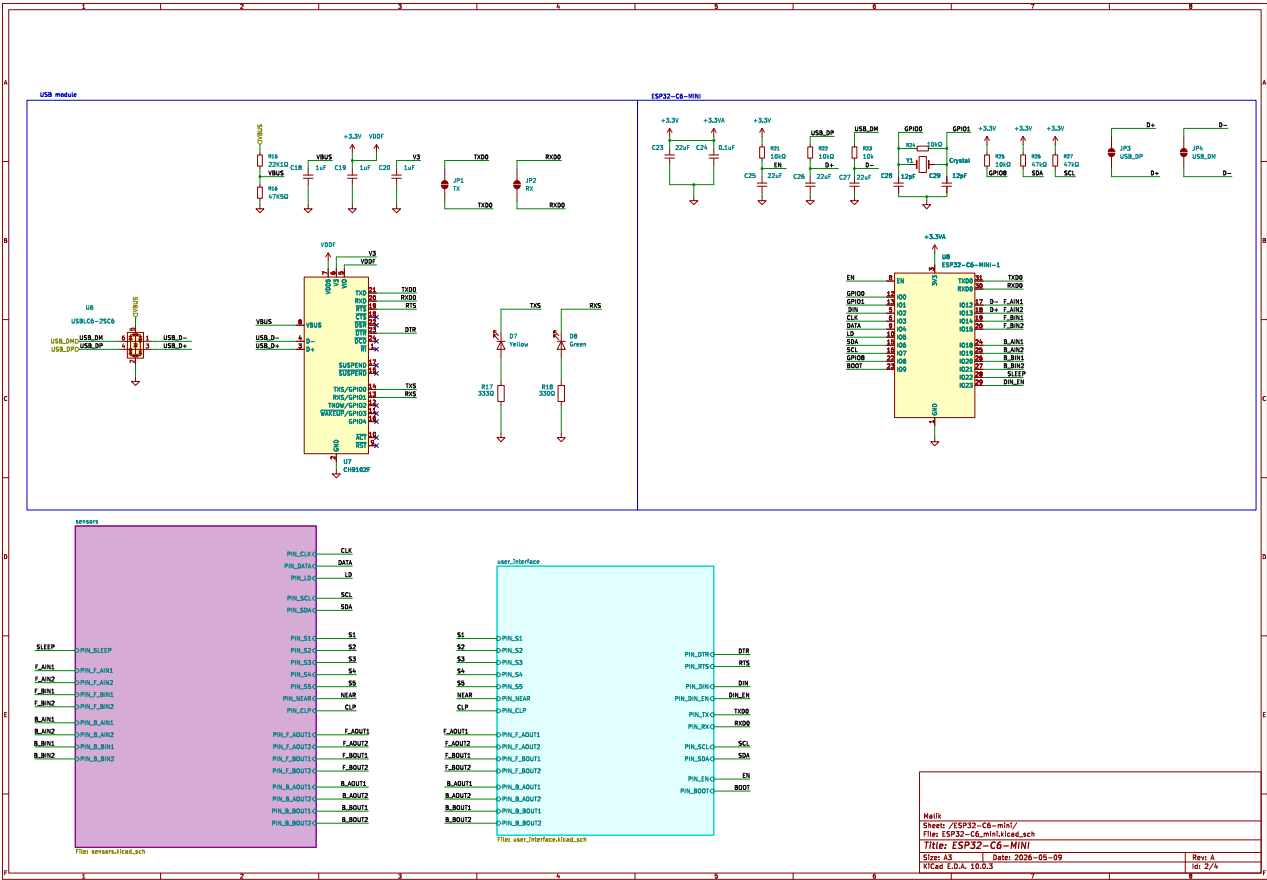
- **Knoppen & Auto-Program Circuit (Boot and Enable):** * Twee fysieke drukknoppen (`SW1` voor Boot en `SW2` voor EN/Reset) maken het mogelijk om de robot handmatig te resetten of in programmeermodus te dwingen.
 - Daarnaast zit er een automatisch programmeercircuit met twee transistoren (`MMBT2222A`) op. Dit zorgt ervoor dat de computer de robot via de `DTR` en `RTS` lijnen automatisch kan resetten en flashen zonder dat je fysiek op de knoppen hoeft te drukken.
- **Externe Connectors (UART & I2C Headers):** Bovenaan bevinden zich twee interfaces (`J7` voor UART en `J8` voor I2C). Hiermee kunnen eenvoudig externe modules worden aangesloten, zoals een OLED-scherm, een Bluetooth-module of een extra sensorbordje, zonder de PCB opnieuw te hoeven ontwerpen.
- **Power Outlets & Connectors:** Onderaan zien we de fysieke connectors (`J5` en `J6`) die de verbindingen leggen naar de buitenwereld, zoals de uitgangen naar de motoren (`DRV_BOUT`, `DRV_FOUT`) en de header naar de 5-weg lijnsensoren (`5way Tracing`).

3. SCHEMA & CIRCUIT ONTWERP

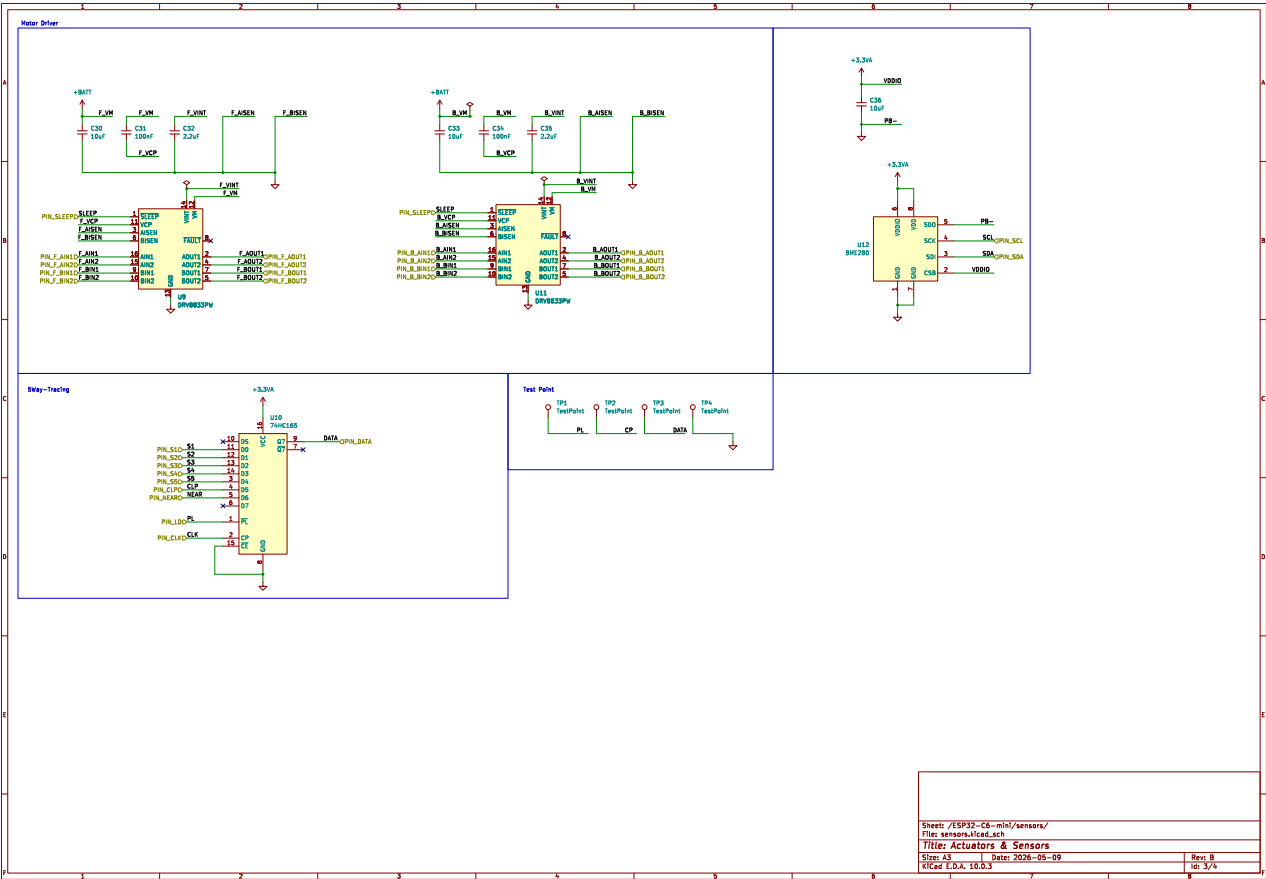
3.1 Pagina 1: Voedingcircuit (Power Subsystem)



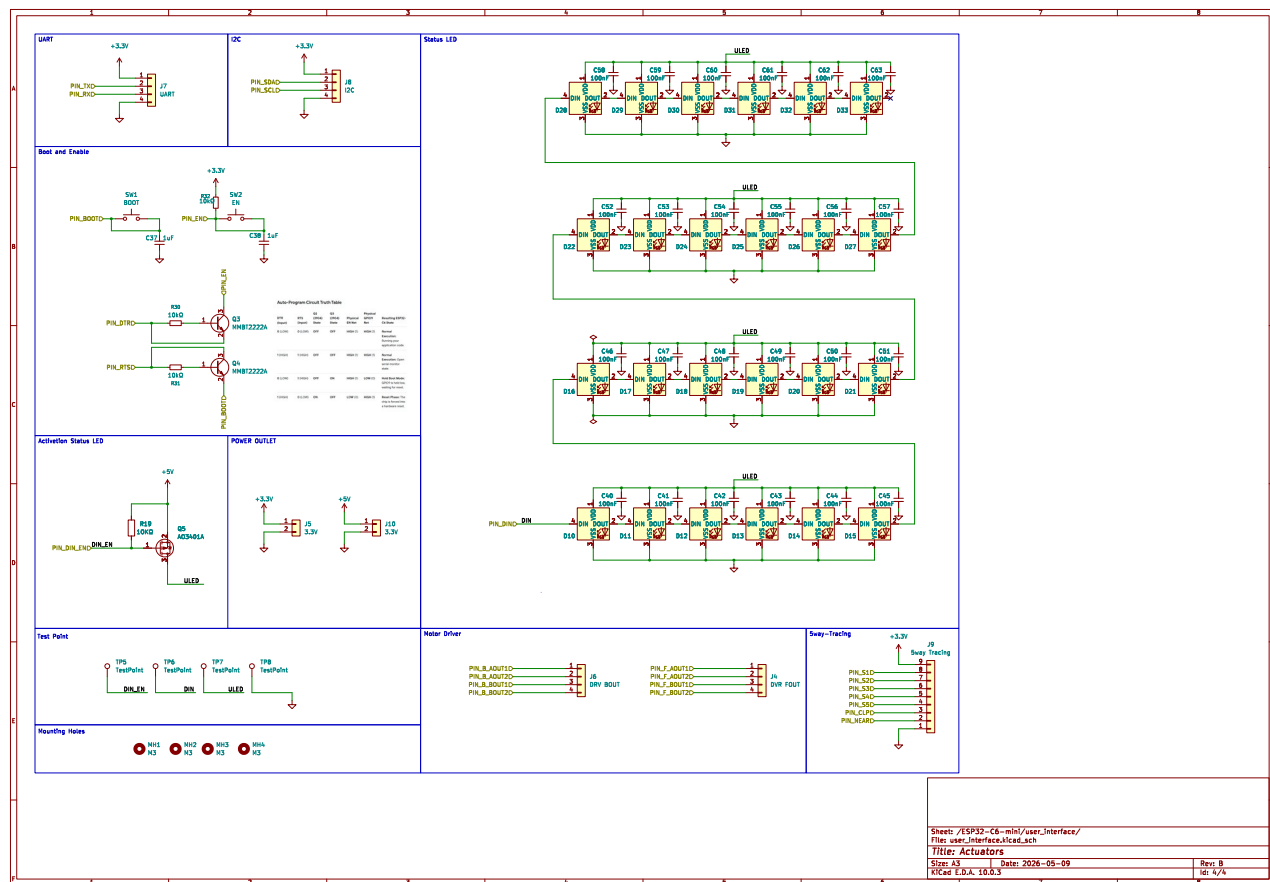
3.2 Pagina 2: Centraal Controlecentrum (ESP32-C6-mini)



3.3 Pagina 3: Sensoraansturing & Motor Drivers



3.4 Pagina 4: Gebruikersinterface & Connectiviteit



3.5 Architectuur van het Schema

Het circuit maakt gebruik van een modulaire opzet waarbij de ESP32-C6-MINI-1 microcontroller centraal staat. De belangrijkste ontwerpaspecten van dit schema zijn:

- I/O Uitbreiding via Schuifregister (74HC165): Om pinnen op de ESP32-C6 te besparen, worden de 5 infrarood-lijsensoren niet individueel ingelezen. In plaats daarvan verzamelt het 74HC165 schuifregister de parallelle statussen en stuurt ze via een seriële datalijn naar de microcontroller.
- Auto-Programmering via USB: Dankzij de CH343F USB-naar-serieel omzetter en het automatische resetcircuit met transistoren (MMBT2222A) kan de robot direct vanuit de Arduino IDE geprogrammeerd en gereset worden zonder fysieke knoppen in te drukken.

- Geïsoleerde Power Rails & Vermogensbeheer: * De 3.3V lijn (via de AMS1117) voedt uitsluitend de logische circuits (ESP32-C6 en de sensoren) voor een storingsvrij signaal.
- De 5V lijn (via de TPS562200 Buck) voedt de grote matrix van WS2812B RGB-status-LED's. Om sluipverbruik te voorkomen, kan deze LED-voeding via een MOSFET (A03416A) volledig worden uitgeschakeld.
- De Directe Batterijlijn (+BATT) voedt de DRV8833 motor drivers rechtstreeks om maximaal vermogen aan de DC-motoren te leveren zonder spanningsregelaars te overbelasten.

3.6 Componentenlijst (BOM - Bill of Materials)

Component	Functie	Interface / Protocol
ESP32-C6-MINI-1	Hoofdprocessor, berekening PID-algoritme & Wi-Fi/Bluetooth	Microcontroller
CH343F	USB-naar-serieel omzetter voor programmering	USB / UART (TXD/RXD)
DRV8833PW (2x)	Dual H-brug motor drivers voor aansturing van 4 DC-motoren	PWM (Pulse Width Modulation)
74HC165	8-bit parallel-in/serial-out schuifregister voor lijnsensoren	Serieel (DATA, CLK, PL)
BME280	Omgevingssensor (Temperatuur, Luchtvochtigheid, Luchtdruk)	I2C (SDA / SCL)
TP5311	Lithium-ion batterijlader IC met status-LED's	Analoog / Voeding
HY2120	BMS (Battery Management System) voor celbeveiliging	Hardwarematig via MOSFET's

TPS562200	Synchrone Buck-regelaar (Stabiele 5V-lijn)	Schakelende voeding
AMS1117-3.3	Lineaire LDO-regelaar (Schone 3.3V-lijn)	Lineaire voeding
WS2812B / SK6812	Adresseerbare RGB status-LED's (Matrix/Strip)	Single-wire Digitaal (PIN_ULED)
AO3416A	N-Channel MOSFET voor stroombesparing LED-matrix	Digitaal I/O (ULED_GND)

4. Het DASHBOARD (GRAFANA VISUALISATIE)

Om de prestaties en de status van de lijnvolgende robot in real-time te bewaken, worden de verzamelde gegevens draadloos doorgestuurd naar een centraal Grafana-dashboard. Dit dashboard fungeert als de controlekamer voor de operator en visualiseert de telemetrie van de robot via overzichtelijke grafieken en meters.

Binnen dit project worden er twee kritieke datastromen visueel weergegeven:

4.1 Batterijstatus (Fuel Gauge)

De batterijspanning, die via het ingebouwde spanningsdeler-circuit op de PCB wordt gemeten door de ESP32-C6, wordt live naar het dashboard gestuurd.

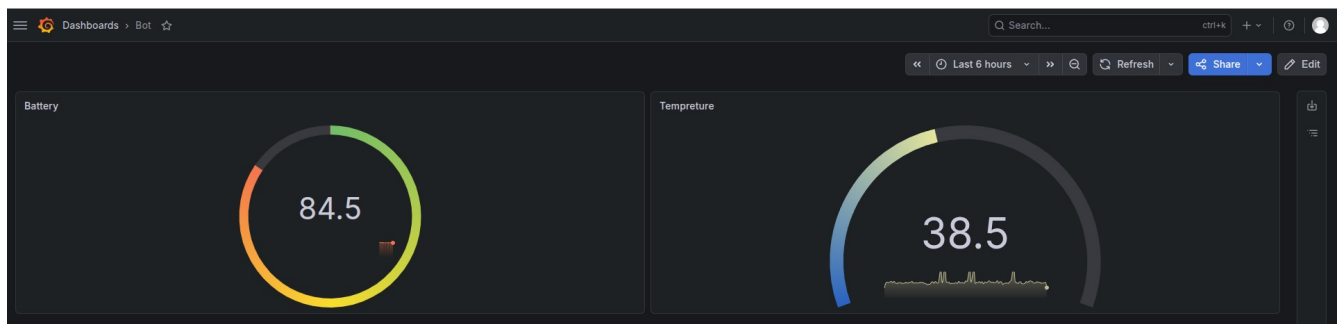
- Visualisatie: Een duidelijke *Gauge widget* (een ronde accumeter of batterijbalk) die van kleur verandert (groen, oranje, rood) op basis van de actuele accuspanning.
- Doel: De operator ziet in één oogopslag hoe vol de lithium-ion batterij nog is. Hierdoor kan tijdig worden ingegrepen voordat het BMS-beveiligingscircuit (HY2120) de robot uitschakelt vanwege een te lage spanning.

4.2 Omgevingstemperatuur (BME280 Telemetrie)

De meetwaarden van de BME280 omgevingssensor worden continu doorgegeven aan de server.

- Visualisatie: Een *Time Series* grafiek die het verloop van de temperatuur (en optioneel de luchtvochtigheid en luchtdruk) over de tijd laat zien.
- Doel: Hiermee kan worden gemonitord of de robot in een gezonde omgeving rijdt. Grote temperatuurschommelingen in de fabriekshal of een plotselinge stijging van de warmte rondom de robot kunnen hiermee direct worden opgemerkt.

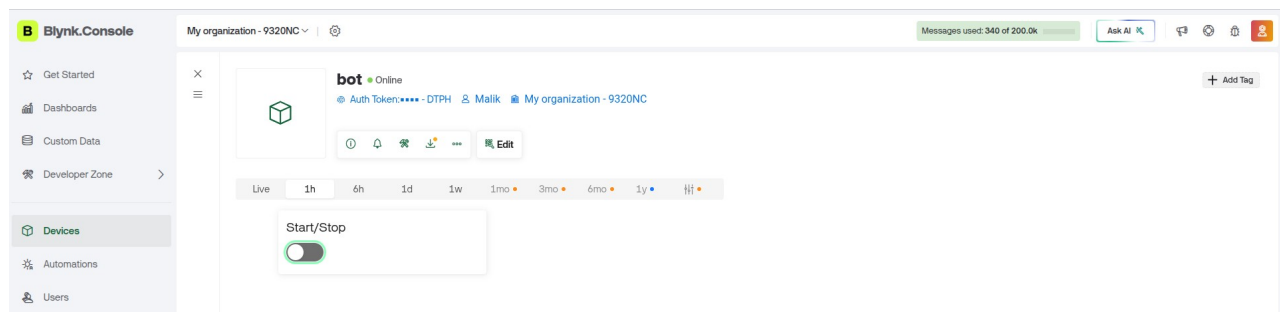
Hoe de data daar komt: De ESP32-C6 maakt gebruik van zijn ingebouwde Wi-Fi om de sensorwaarden via het MQTT-protocol te publiceren. Een centrale database (zoals InfluxDB) slaat deze tijdgebonden data op, waarna Grafana de cijfers direct omzet in de live-grafieken op het scherm.



4.3 Externe Besturing (Blynk Remote Switch)

Naast het monitoren van data in Grafana, heeft de operator ook de mogelijkheid om de robot op afstand aan te sturen. Hiervoor is een mobiele interface ingericht via het Blynk IoT-platform.

- Visualisatie & Bediening: Een virtuele start/stop-knop (Switch) op een smartphone of tablet.
- Doel: Met deze digitale schakelaar kan de operator de robot draadloos starten of een directe noodstop (Stop) geven. Zodra de knop in de app wordt omgezet, stuurt de Blynk-cloud direct een signaal naar de ESP32-C6 via Wi-Fi. De microcontroller activeert of deactiveert vervolgens direct de SLEEP pinnen van de DRV8833 motor drivers, waardoor de motoren direct stoppen of gaan rijden.



5. PROGRAMMACODE

ESP32_C6_Line_Following_Bot_Telemetry_Remote_Button.ino

```

1  #define BLYNK_TEMPLATE_ID "TMPL5pyPNEr_b"
2  #define BLYNK_TEMPLATE_NAME "NetworkBlink"
3  #define BLYNK_AUTH_TOKEN "M48Pm2mKoycKpG493HZNSSvn01gZDTPH"
4  #define BLYNK_PRINT Serial
5
6  #include <WiFi.h>
7  #include <WiFiClient.h>
8  #include <BlynkSimpleEsp32.h>
9  #include <PubSubClient.h>
10 #include <ArduinoJson.h>
11 #include <Adafruit_NeoPixel.h>
12
13 // --- PHYSICAL HARDWARE PIN REGISTRY ---
14 const int motorLeftFwd = 0;
15 const int motorLeftRev = 1;
16 const int motorRightFwd = 2;
17 const int motorRightRev = 3;
18
19 const int pinS1 = 18;
20 const int pinS2 = 19;
21 const int pinS3 = 20;
22 const int pinS4 = 21;
23 const int pinS5 = 22;
24 const int pinNear = 12; // CHANGED: Moved from problematic Pin 7 to safe Pin 12!
25
26 #define BATT_ADC_PIN 4
27 #define DIVIDER_RATIO 2.0f
28
29 #define LED_PIN 15
30 #define NUM_LEDS 13
31 #define BRIGHTNESS 25
32 #define BLYNK_LED_PIN 2
33
34 // --- INSTANCE DECLARATIONS ---
35 Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
36 WiFiClient espClient;
37 PubSubClient client(espClient);
38
39 char ssid[] = "mnet-home";
40 char pass[] = "mnet*2026";
41 const char* mqtt_server = "192.168.0.17";
42 const int mqtt_port = 1883;
43 const char* mqtt_topic = "robot/telemetry";
44

```

```

44
45 // --- GLOBAL VARIABLES ---
46 bool robotEnabled = false;
47 int globalSocPercent = 100;
48 float batteryVoltage = 0.0f;
49
50 unsigned long lastMsg = 0;
51 unsigned long lastLedUpdate = 0;
52 unsigned long lastSerialPrint = 0;
53 unsigned long lastBlynkCheck = 0;
54 const int animationInterval = 100;
55 int animationFrame = 0;
56
57 enum RobotState { MOVING_FORWARD, TURNING_LEFT, TURNING_RIGHT, PIVOT_LEFT, PIVOT_RIGHT, OBSTACLE_STOP, LINE_LOST_STOP, BL
58 RobotState currentRobotState = BLYNK_DISABLED_STOP;
59
60 // --- MOTOR CONTROL FUNCTIONS ---
61 void moveForward() {
62     digitalWrite(motorLeftFwd, HIGH); digitalWrite(motorLeftRev, LOW);
63     digitalWrite(motorRightFwd, HIGH); digitalWrite(motorRightRev, LOW);
64 }
65 void turnLeftGently() {
66     digitalWrite(motorLeftFwd, LOW); digitalWrite(motorLeftRev, LOW);
67     digitalWrite(motorRightFwd, HIGH); digitalWrite(motorRightRev, LOW);
68 }
69 void pivotLeftSharp() {
70     digitalWrite(motorLeftFwd, LOW); digitalWrite(motorLeftRev, HIGH);
71     digitalWrite(motorRightFwd, HIGH); digitalWrite(motorRightRev, LOW);
72 }
73 void turnRightGently() {
74     digitalWrite(motorLeftFwd, HIGH); digitalWrite(motorLeftRev, LOW);
75     digitalWrite(motorRightFwd, LOW); digitalWrite(motorRightRev, LOW);
76 }
77 void pivotRightSharp() {
78     digitalWrite(motorLeftFwd, HIGH); digitalWrite(motorLeftRev, LOW);
79     digitalWrite(motorRightFwd, LOW); digitalWrite(motorRightRev, HIGH);
80 }
81 void stopRobot() {
82     digitalWrite(motorLeftFwd, LOW); digitalWrite(motorLeftRev, LOW);
83     digitalWrite(motorRightFwd, LOW); digitalWrite(motorRightRev, LOW);
84 }

```

```

85
86 // --- BLYNK ---
87 BLYNK_CONNECTED() {
88   Blynk.syncVirtual(V0);
89 }
90
91 BLYNK_WRITE(V0) {
92   int value = param.asInt();
93   digitalWrite(BLYNK_LED_PIN, value);
94
95   if (value == 1) {
96     robotEnabled = true;
97   } else {
98     robotEnabled = false;
99     stopRobot();
100     currentRobotState = BLYNK_DISABLED_STOP;
101   }
102 }
103
104 void verifyMqttConnection() {
105   if (!client.connected()) {
106     client.connect("ESP32Client-Robot");
107   }
108 }
109
110 void setup() {
111   Serial.begin(115200);
112
113   pinMode(motorLeftFwd, OUTPUT); pinMode(motorLeftRev, OUTPUT);
114   pinMode(motorRightFwd, OUTPUT); pinMode(motorRightRev, OUTPUT);
115   pinMode(BLYNK_LED_PIN, OUTPUT);
116
117   pinMode(pinS1, INPUT); pinMode(pinS2, INPUT); pinMode(pinS3, INPUT);
118   pinMode(pinS4, INPUT); pinMode(pinS5, INPUT);
119
120   // CHANGED: Added INPUT_PULLUP to stabilize the sensor readings
121   pinMode(pinNear, INPUT_PULLUP);
122
123   analogSetAttenuation(ADC_11db);
124
125   strip.begin();
126   strip.setBrightness(BRIGHTNESS);
127   strip.clear();
128   strip.show();
129
130   Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
131   client.setServer(mqtt_server, mqtt_port);
132 }

```

```

133
134 void loop() {
135     Blynk.run();
136     client.loop();
137
138     unsigned long now = millis();
139
140     if (now - lastBlynkCheck >= 5000) {
141         lastBlynkCheck = now;
142         if (Blynk.connected()) {
143             verifyMqttConnection();
144         }
145     }
146
147     // --- BATTERY MONITORING ---
148     uint32_t pinMilliVolts = analogReadMilliVolts(BATT_ADC_PIN);
149     batteryVoltage = (pinMilliVolts / 1000.0f) * DIVIDER_RATIO;
150     globalSocPercent = constrain(((batteryVoltage - 3.00f) / (4.20f - 3.00f)) * 100.0f, 0, 100);
151
152     // --- SENSOR READS ---
153     int s1 = digitalRead(pinS1);
154     int s2 = digitalRead(pinS2);
155     int s3 = digitalRead(pinS3);
156     int s4 = digitalRead(pinS4);
157     int s5 = digitalRead(pinS5);
158     int nearState = digitalRead(pinNear);
159
160     // --- NAVIGATION LOGIC ---
161     if (!robotEnabled) {
162         stopRobot();
163         currentRobotState = BLYNK_DISABLED_STOP;
164     }
165     // CHANGED: Switched nearState trigger to LOW.
166     // (Standard IR obstacle modules output 1 normally, and flip to 0 when they see something)
167     else if (nearState == LOW) {
168         stopRobot();
169         currentRobotState = OBSTACLE_STOP;
170     }
171     else if ((s1 == LOW && s2 == LOW && s3 == HIGH && s4 == LOW && s5 == LOW) ||
172             (s1 == LOW && s2 == HIGH && s3 == HIGH && s4 == HIGH && s5 == LOW)) {
173         moveForward();
174         currentRobotState = MOVING_FORWARD;
175     }
176     else if (s2 == HIGH && s4 == LOW) {
177         turnLeftGently();
178         currentRobotState = TURNING_LEFT;
179     }
180     else if (s1 == HIGH) {
181         pivotLeftSharp();
182         currentRobotState = PIVOT_LEFT;
183     }
184     else if (s4 == HIGH && s2 == LOW) {
185         turnRightGently();
186         currentRobotState = TURNING_RIGHT;
187     }

```

```

188 ✓ else if (s5 == HIGH) {
189     pivotRightSharp();
190     currentRobotState = PIVOT_RIGHT;
191 }
192 ✓ else {
193     stopRobot();
194     currentRobotState = LINE_LOST_STOP;
195 }
196
197 updateStatusIndicators();
198 printLiveTelemetry(s1, s2, s3, s4, s5, nearState);
199
200 // --- MQTT TELEMETRY SYSTEM ---
201 ✓ if (now - lastMsg > 5000) {
202     lastMsg = now;
203
204     JsonDocument doc;
205     doc["device_id"] = "device_01";
206     doc["status"] = (robotEnabled) ? "discharging" : "charging";
207     doc["remaining_mah"] = map(globalSocPercent, 0, 100, 0, 2400);
208     doc["soc_percent"] = globalSocPercent;
209     doc["voltage_v"] = (batteryVoltage / 5.6) * 100;
210     doc["current_a"] = random(50, 220) / 100.0;
211     doc["soh_percent"] = random(970, 1000) / 10.0;
212     doc["temperature_c"] = random(240, 350) / 100.0;
213     doc["robot_state"] = (int)currentRobotState;
214     doc["blynk_enabled"] = robotEnabled;
215
216     char buffer[256];
217     serializeJson(doc, buffer);
218
219 ✓     if (client.connected()) {
220         client.publish(mqtt_topic, buffer);
221     }
222 }
223 }
224
225 // --- TELEMETRY LOGGING ---
226 ✓ void printLiveTelemetry(int s1, int s2, int s3, int s4, int s5, int near) {
227 ✓     if (millis() - lastSerialPrint >= 1000) {
228         lastSerialPrint = millis();
229 ✓         Serial.printf("SENSORS: [S1:%d S2:%d S3:%d S4:%d S5:%d] | NEAR:%d | BATT: %.2fV (%d%%)\n",
230             s1, s2, s3, s4, s5, near, batteryVoltage, globalSocPercent);
231     }
232 }

```

GNU nano 8.4

```
import json
from influxdb import InfluxDBClient
import paho.mqtt.client as mqtt

# Configuration
MQTT_BROKER = "localhost"
MQTT_PORT = 1883
MQTT_TOPIC = "robot/telemetry"

INFLUX_HOST = "localhost"
INFLUX_PORT = 8086
INFLUX_USER = "botuser"
INFLUX_PASS = "robot2026"
INFLUX_DB = "robot"

# Initialize InfluxDB Client
db_client = InfluxDBClient(
    host=INFLUX_HOST,
    port=INFLUX_PORT,
    username=INFLUX_USER,
    password=INFLUX_PASS,
    database=INFLUX_DB
)

# Callback when connecting to the MQTT broker
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print(f"Connected to MQTT Broker! Subscribing to: {MQTT_TOPIC}")
        client.subscribe(MQTT_TOPIC)
    else:
        print(f"MQTT Connection failed with code {rc}")

# Callback when an MQTT message is received
def on_message(client, userdata, msg):
    try:
        # Decode JSON payload from MQTT
        payload = json.loads(msg.payload.decode('utf-8'))
        print(f"Received data: {payload}")
```

```

# Build InfluxDB JSON structure
json_body = [
    {
        "measurement": "battery_metrics",
        "tags": {
            "device_id": payload.get("device_id", "unknown_device"),
            "status": payload.get("status", "unknown")
        },
        "fields": {
            "current_a": float(payload["current_a"]),
            "remaining_mah": int(payload["remaining_mah"]),
            "soc_percent": int(payload["soc_percent"]),
            "soh_percent": float(payload["soh_percent"]),
            "temperature_c": float(payload["temperature_c"]),
            "voltage_v": float(payload["voltage_v"])
        }
    }
]

# Write to InfluxDB
db_client.write_points(json_body)
print("Successfully written to InfluxDB.")

except KeyError as e:
    print(f"Data missing expected field: {e}")
except ValueError as e:
    print(f"Data type conversion error: {e}")
except Exception as e:
    print(f"Error handling message: {e}")

if __name__ == "__main__":
    mqtt_client = mqtt.Client()
    mqtt_client.on_connect = on_connect
    mqtt_client.on_message = on_message

    print("Starting MQTT to InfluxDB Bridge...")
    try:
        mqtt_client.connect(MQTT_BROKER, MQTT_PORT, 60)
        mqtt_client.loop_forever()
    except KeyboardInterrupt:
        print("\nDisconnecting bridge...")
        mqtt_client.disconnect()

```

6. DATASHEET

ESP32-C6-MINI-1 ESP32-C6-MINI-1U

Datasheet Version 1.4

Module that supports 2.4 GHz Wi-Fi 6 (802.11ax), Bluetooth® 5 (LE), Zigbee and Thread (802.15.4)

Built around ESP32-C6 series of SoCs, 32-bit RISC-V single-core microprocessor

Flash up to 8 MB in chip package

22 GPIOs, rich set of peripherals

On-board PCB antenna or external antenna connector



ESP32-C6-MINI-1



ESP32-C6-MINI-1U

9 Peripheral Schematics

This is the typical application circuit of the module connected with peripheral components (for example, power supply, antenna, reset button, JTAG interface, and UART interface).

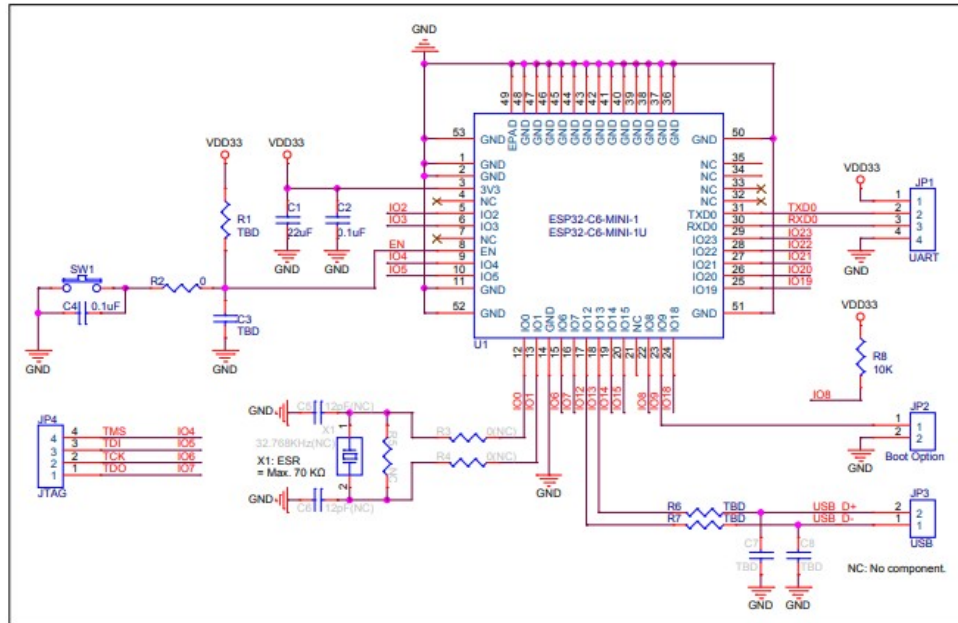


Figure 9-1. Peripheral Schematics

- Soldering the EPAD to the ground of the base board is not a must, however, it can optimize thermal performance. If you choose to solder it, please apply the correct amount of soldering paste. Too much soldering paste may increase the gap between the module and the baseboard. As a result, the adhesion between other pins and the baseboard may be poor.
- To ensure that the power supply to the ESP32-C6 chip is stable during power-up, it is advised to add an RC delay circuit at the EN pin. The recommended setting for the RC delay circuit is usually $R = 10\text{ k}\Omega$ and $C = 1\text{ }\mu\text{F}$. However, specific parameters should be adjusted based on the power-up timing of the module and the power-up and reset sequence timing of the chip. For ESP32-C6's power-up and reset sequence timing diagram, please refer Section 4.5 *Chip Power-up and Reset*.

3 Pin Definitions

3.1 Pin Layout

The pin diagram below shows the approximate location of pins on the module. For the actual diagram drawn to scale, please refer to Figure 10.1 *Module Dimensions*.

The pin diagram is applicable for ESP32-C6-MINI-1 and ESP32-C6-MINI-1U, but the latter has no keepout zone.

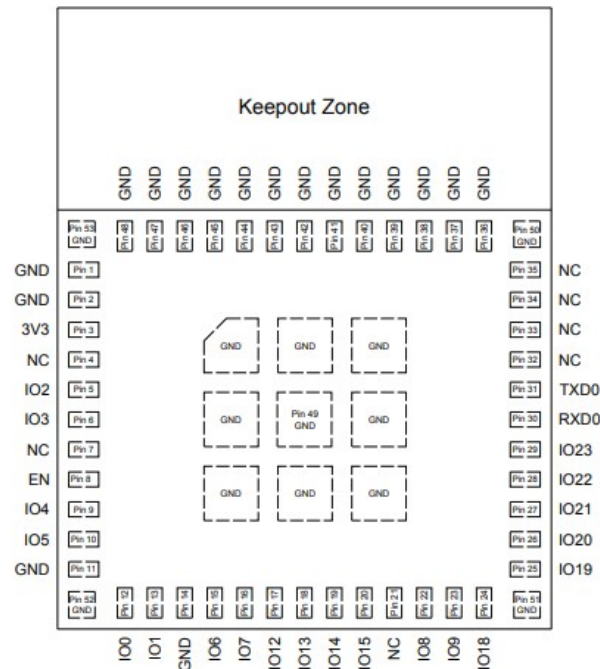


Figure 3-1. Pin Layout (Top View)

3.2 Pin Description

The module has 53 pins. See pin definitions in Table 3-1 *Pin Definitions*.

For peripheral pin configurations, please refer to [ESP32-C6 Series Datasheet](#).

Table 3-1. Pin Definitions

Table 3-1 – cont'd from previous page

Name	No.	Type ¹	Function
NC	4	—	NC
IO2	5	I/O/T	GPIO2, LP_GPIO2, LP_UART_RTSN, ADC1_CH2, FSPiQ
IO3	6	I/O/T	GPIO3, LP_GPIO3, LP_UART_CTSN, ADC1_CH3
NC	7	—	NC
EN	8	I	High: on, enables the chip. Low: off, the chip powers off. Note: Do not leave the EN pin floating.
IO4	9	I/O/T	MTMS, GPIO4, LP_GPIO4, LP_UART_RXD, ADC1_CH4, FSPiHD
IO5	10	I/O/T	MTDI, GPIO5, LP_GPIO5, LP_UART_TXD, ADC1_CH5, FSPiWP
IO0	12	I/O/T	GPIO0, XTAL_32K_P, LP_GPIO0, LP_UART_DTRN, ADC1_CH0
IO1	13	I/O/T	GPIO1, XTAL_32K_N, LP_GPIO1, LP_UART_DSRN, ADC1_CH1
IO6	15	I/O/T	MTCK, GPIO6, LP_GPIO6, LP_I2C_SDA, ADC1_CH6, FSPiCLK
IO7	16	I/O/T	MTDO, GPIO7, LP_GPIO7, LP_I2C_SCL, FSPiD
IO12	17	I/O/T	GPIO12, USB_D-
IO13	18	I/O/T	GPIO13, USB_D+
IO14	19	I/O/T	GPIO14
IO15	20	I/O/T	GPIO15
NC	21	—	NC
IO8	22	I/O/T	GPIO8
IO9	23	I/O/T	GPIO9
IO18	24	I/O/T	GPIO18, SDIO_CMD, FSPiCS2
IO19	25	I/O/T	GPIO19, SDIO_CLK, FSPiCS3
IO20	26	I/O/T	GPIO20, SDIO_DATA0, FSPiCS4
IO21	27	I/O/T	GPIO21, SDIO_DATA1, FSPiCS5
IO22	28	I/O/T	GPIO22, SDIO_DATA2
IO23	29	I/O/T	GPIO23, SDIO_DATA3
RXD0	30	I/O/T	UORXD, GPIO17, FSPiCS1
TXD0	31	I/O/T	UOTXD, GPIO16, FSPiCS0
NC	32	—	NC
NC	33	—	NC
NC	34	—	NC
NC	35	—	NC

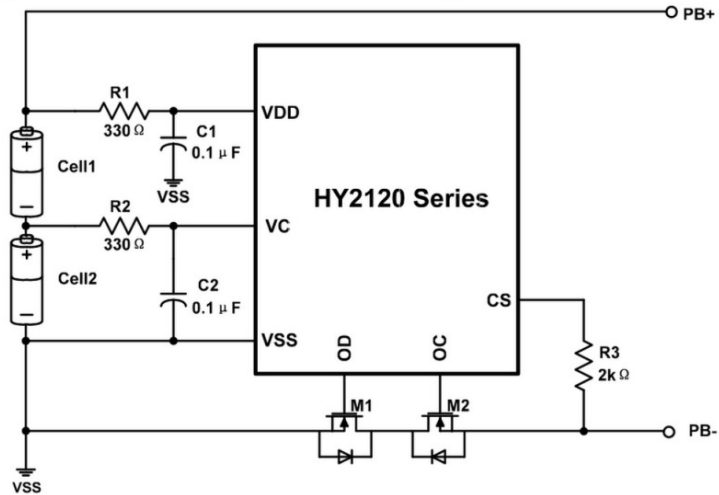
¹ P: power supply; I: input; O: output; T: high impedance.

HY2120

2-Cell Lithium-ion/Lithium Polymer Battery Packs Protection ICs



10. Battery Protection IC Connection Example

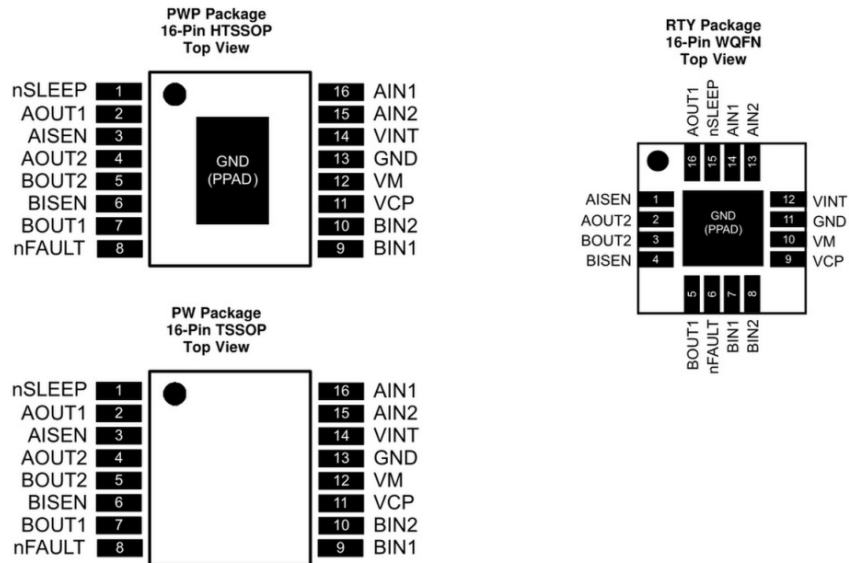


Symbol	Device Name	Purpose	Min.	Typ.	Max.	Remark
R1	Resistor	limit current, stabilize VDD and strengthen ESD protection	100Ω	330Ω	470Ω	*1
R2	Resistor	limit current, stabilize VC and strengthen ESD protection	100Ω	330Ω	470Ω	*1
R3	Resistor	limit current	1 kΩ	2kΩ	4kΩ	*2
C1	Capacitor	Filter, stabilize VDD	0.01μF	0.1μF	1.0μF	*3
C2	Capacitor	Filter, stabilize VDD	0.01μF	0.1μF	1.0μF	*3
M1	N-MOSFET	Discharge control	-	-	-	*4
M2	N-MOSFET	Charge control	-	-	-	*5

*1. If R1 or R2 connects with an over-spec resistor, battery accuracy may be influenced due to R1 or R2 voltage drop that caused by current consumption. When a charger is connected in reversed, the current flows from the charger to the IC. At this time, if R1 or R2 is too high, the voltage between VDD pin and VSS pin may exceed the absolute maximum rating.

*2. If R3 connects with an over-spec resistor, the charging current may not be cut off when a high-voltage charger is connected. Please select as large a resistor as possible to control current

5 Pin Configuration and Functions



Pin Functions

PIN		I/O ⁽¹⁾	DESCRIPTION	EXTERNAL COMPONENTS OR CONNECTIONS	
NAME	WQFN				HTSSOP, TSSOP
POWER AND GROUND					
GND	11 PPAD	13	—	Device ground. HTSSOP package has PowerPAD.	Both the GND pin and device PowerPAD must be connected to ground.
VINT	12	14	—	Internal supply bypass	Bypass to GND with 2.2-μF, 6.3-V capacitor.
VM	10	12	—	Device power supply	Connect to motor supply. A 10-μF (minimum) ceramic bypass capacitor to GND is recommended.
VCP	9	11	IO	High-side gate drive voltage	Connect a 0.01-μF, 16-V (minimum) X7R ceramic capacitor to VM.
CONTROL					
AIN1	14	16	I	Bridge A input 1	Logic input controls state of AOUT1. Internal pulldown.
AIN2	13	15	I	Bridge A input 2	Logic input controls state of AOUT2. Internal pulldown.
BIN1	7	9	I	Bridge B input 1	Logic input controls state of BOUT1. Internal pulldown.
BIN2	8	10	I	Bridge B input 2	Logic input controls state of BOUT2. Internal pulldown.
nSLEEP	15	1	I	Sleep mode input	Logic high to enable device, logic low to enter low-power sleep mode and reset all internal logic. Internal pulldown.

(1) I = Input, O = Output, OZ = Tri-state output, OD = Open-drain output, IO = Input/output



DRV8833

www.ti.com

SLVSAR1E – JANUARY 2011 – REVISED JULY 2015

Electrical Characteristics (continued)

 $T_A = 25^\circ\text{C}$ (unless otherwise noted)

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
CURRENT CONTROL					
V_{TRIP}	xISEN trip voltage	160	200	240	mV
t_{BLANK}	Current sense blanking time		3.75		μs
SLEEP MODE					
t_{WAKE}	Start-up time	nSLEEP inactive high to H-bridge on		1	ms

6.6 Typical Characteristics

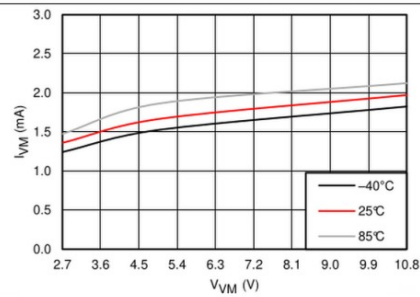


Figure 1. Operating Current

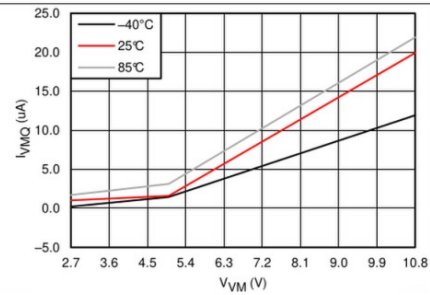
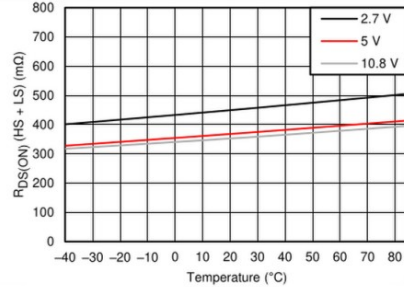
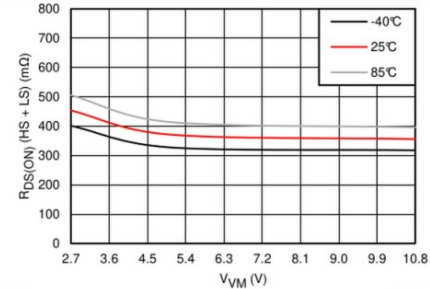


Figure 2. Sleep Current

Figure 3. $R_{DS(on)}$ (HS + LS)Figure 4. $R_{DS(on)}$ (HS + LS)

achieve higher current.

8.2 Typical Application

The two H-bridges in the DRV8833 can be connected in parallel for double the current of a single H-bridge. The internal dead time in the DRV8833 prevents any risk of cross-conduction (shoot-through) between the two bridges due to timing differences between the two bridges. [Figure 7](#) shows the connections.

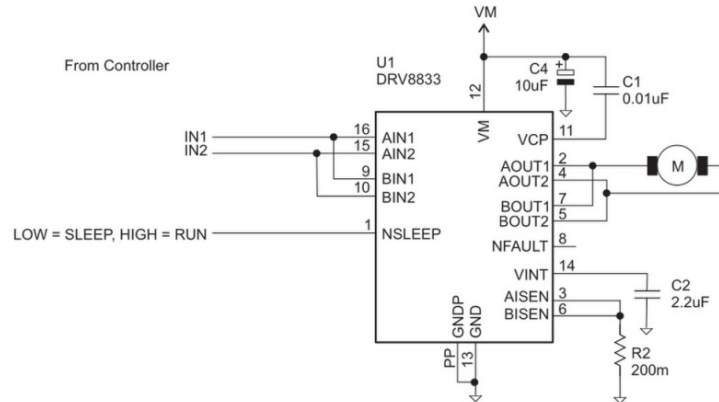


Figure 7. Parallel Mode

8.2.1 Design Requirements

Table 5. Design Parameters

DESIGN PARAMETER	REFERENCE	EXAMPLE VALUE
Motor voltage	V_M	10 V
Motor RMS current	I_{RMS}	0.8 A
Motor start-up current	I_{START}	2 A
Motor current trip point	I_{TRIP}	2.5 A

8.2.2 Detailed Design Procedure

8.2.2.1 Motor Voltage

The motor voltage to use will depend on the ratings of the motor selected and the desired RPM. A higher voltage spins a brushed DC motor faster with the same PWM duty cycle applied to the power FETs. A higher voltage also increases the rate of current change through the inductive motor windings.

8.2.2.2 Motor Current Trip Point

When the voltage on pin xISEN exceeds V_{TRIP} (0.2 V), current regulation is activated. The R_{ISENSE} resistor should be sized to set the desired I_{CHOP} level.

- Chip temperature protection, overcurrent, undervoltage protection
- Battery temperature protection, reverse battery shutdown, short circuit protection
- Switching frequency of 400KHz, and more available inductance 20uH
- PWR_ON- power, battery-powered switching control
- Charging voltage control accuracy of less than 1%
- Trickle, constant current, constant voltage charging three sections, a battery protected phone
- QFN16 4mm * 4mm using ultra small package
- Portable equipment, various chargers
- Smart phone, PDA, mobile cellular phone
- MP4, MP5 players, tablet computers
- HM
- electrical tools

typical application

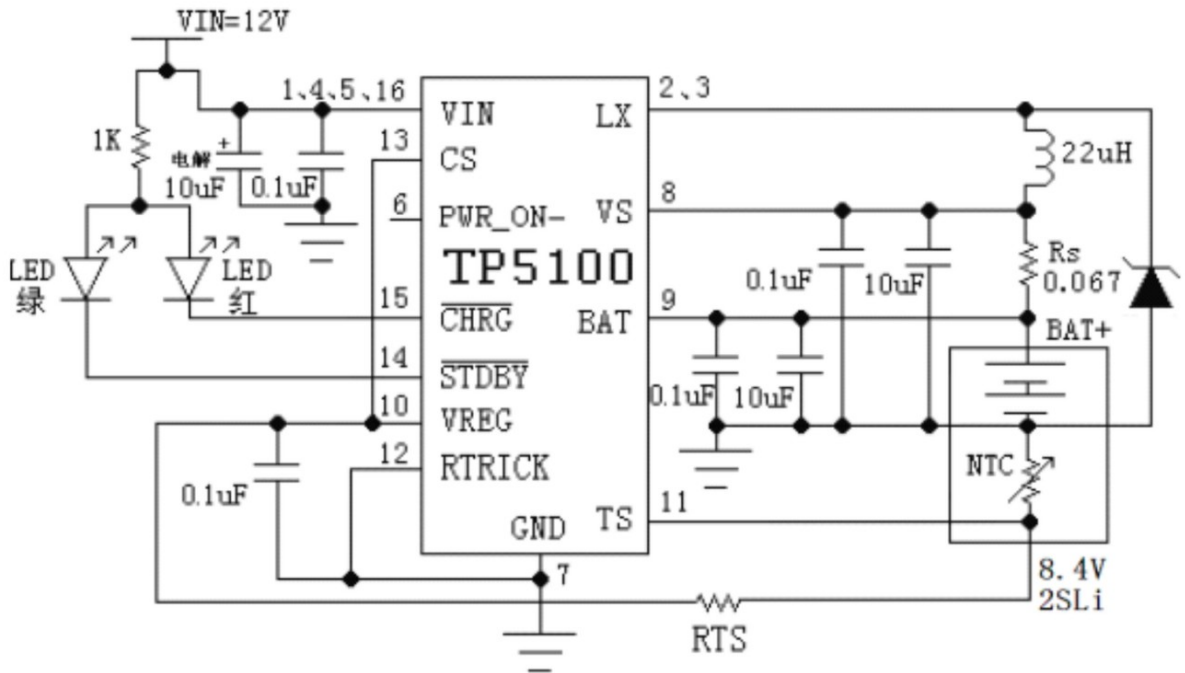


FIG 2 TP5100 lithium ion battery is 8.4V 1.5A double charge (precharge 150MA) Application schematic

TPS5420 2A, Wide Input Range, Step-Down Converter

1 Features

- Wide input voltage range: 5.5V to 36V
- Up to 2A continuous (3A peak) output current
- High efficiency up to 95% enabled by 110mΩ integrated MOSFET switch
- Wide output voltage range: adjustable down to 1.22V with 1.5% initial accuracy
- Internal compensation minimizes external parts count
- Fixed 500kHz switching frequency for small filter size
- Improved line regulation and transient response by input voltage feed forward
- System protected by over current limiting, overvoltage protection and thermal shutdown
- -40°C to 125°C operating junction temperature range
- Available in small 8-pin SOIC package

2 Applications

- Consumer: set-top box, DVD, LCD displays
- Industrial and car audio power supplies
- Battery chargers, high power LED supply
- 12V, 24V distributed power systems

3 Description

The TPS5420 is a high-output-current PWM converter that integrates a low resistance high side N-channel MOSFET. Included on the substrate with the listed features is a high performance voltage error amplifier that provides tight voltage regulation accuracy under transient conditions; an undervoltage-lockout circuit to prevent start-up until the input voltage reaches 5.5V; an internally set slow-start circuit to limit inrush currents; and a voltage feed-forward circuit to improve the transient response. Using the ENA pin, shutdown supply current is reduced to 18μA typically. Other features include an active-high enable, overcurrent limiting, overvoltage protection and thermal shutdown. To reduce design complexity and external component count, the TPS5420 feedback loop is internally compensated.

The TPS5420 device is available in an easy to use 8-pin SOIC package. TI provides evaluation modules and the Designer software tool to aid in quickly achieving high-performance power supply designs to meet aggressive equipment development cycles.

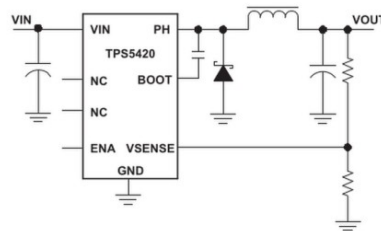
Package Information

PART NUMBER	PACKAGE ⁽¹⁾	PACKAGE SIZE ⁽²⁾
TPS5420	D (SOIC, 8)	4.9mm × 6mm

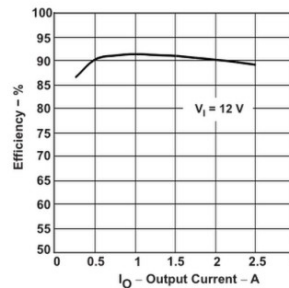
(1) For more information, see [Section 10](#).

(2) The package size (length × width) is a nominal value and includes pins, where applicable.

Simplified Schematic



Efficiency vs Output Current



TPS5420

SLVS642G – APRIL 2006 – REVISED APRIL 2026

**7 Application and Implementation****Note**

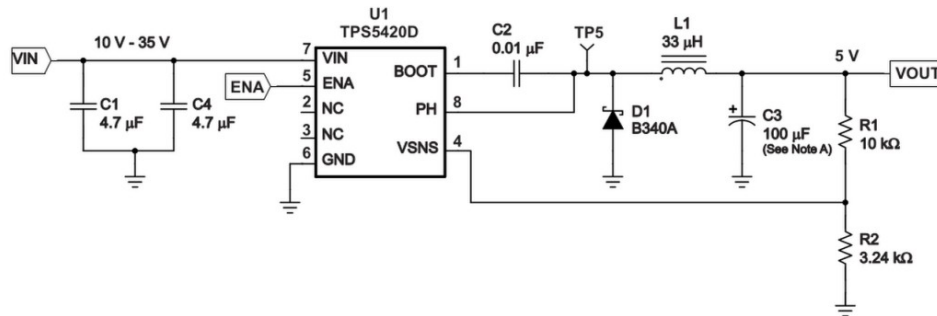
Information in the following applications sections is not part of the TI component specification, and TI does not warrant its accuracy or completeness. TI's customers are responsible for determining suitability of components for their purposes, as well as validating and testing their design implementation to confirm system functionality.

7.1 Application Information

The TPS5420 is a 2A, step-down regulator with an integrated high-side MOSFET. This device is typically used to convert a higher DC voltage to a lower DC voltage with a maximum available output current of 2A. Example applications are: high density point-of-load regulators for set-top box, high power LED supply, and other distributed power systems. Use the following design procedure to select component values for the TPS5420. This procedure illustrates the design of a high frequency switching regulator. Alternatively, use the WEBENCH circuit design and selection simulation services software to generate a complete design. The WEBENCH circuit design and selection simulation services software uses an iterative design procedure and accesses a comprehensive database of components when generating a design.

7.2 Typical Application**7.2.1 Application Circuits**

Figure 7-1 shows the schematic for a typical TPS5420 application. The TPS5420 can provide up to 2A output current at a nominal output voltage of 5V.



A. C3 = Tantalum AVX TPSD107M010R0080

Figure 7-1. Application Circuit, 10V — 35V to 5V



www.ti.com

TPS5420
SLVS642G – APRIL 2006 – REVISED APRIL 2026**7.2.1.1 Design Requirements**

For this design example, use the following as the input parameters:

DESIGN PARAMETER ⁽¹⁾	EXAMPLE VALUE
Input voltage range	10V to 36V
Output voltage	5V
Input ripple voltage	300mV
Output ripple voltage	30mV
Output current rating	2A
Operating frequency	500kHz

(1) As an additional constraint, the design is set up to be small size and low component height.

7.2.1.2 Detailed Design Procedure

The following design procedure can be used to select component values for the TPS5420. Alternately, the Designer Software can be used to generate a complete design. The Designer Software uses an iterative design procedure and accesses a comprehensive database of components when generating a design. This section presents a simplified discussion of the design process.

To begin the design process, a few parameters must be determined. The designer must know the following:

- Input voltage range
- Output voltage
- Input ripple voltage
- Output ripple voltage
- Output current rating
- Operating frequency

7.2.1.2.1 Switching Frequency

The switching frequency for the TPS5420 is internally set to 500kHz. It is not possible to adjust the switching frequency.

7.2.1.2.2 Input Capacitors

The TPS5420 requires an input decoupling capacitor and, depending on the application, a bulk input capacitor. The recommended value for the decoupling capacitor is 10μF. A high quality ceramic type X5R or X7R is required. For some applications, a smaller value decoupling capacitor can be used, if the input voltage and current ripple ratings are not exceeded. The voltage rating must be greater than the maximum input voltage, including ripple. For this design, two 4.7μF capacitors, C1 and C4 are used to allow for smaller 1812 case size to be used while maintaining a 50V rating.

This input ripple voltage can be approximated by Equation 2:

$$\Delta V_{IN} = \frac{I_{OUT(MAX)} \times 0.25}{C_{BULK} \times f_{SW}} + (I_{OUT(MAX)} \times ESR_{MAX}) \quad (2)$$

Where $I_{OUT(MAX)}$ is the maximum load current, f_{SW} is the switching frequency, C_1 is the input capacitor value and ESR_{MAX} is the maximum series resistance of the input capacitor.

The maximum RMS ripple current also needs to be checked. For worst case conditions, this is approximated by Equation 3:

$$I_{CIN} = \frac{I_{OUT(MAX)}}{2} \quad (3)$$

expand high-power input, such as wireless chargers, electric toothbrushes, rechargeable shavers, lithium battery power tools and other applications.

2. Features

! Support 4V to 22V input voltage ! Support

PD3.0/2.0, BC1.2 and other fast charging protocols ! Support

USB Type-C PD, support positive and negative plug detection and automatic

switching ! Support E-Mark simulation, automatic detection of VCONN, support 100W power PD

request ! The request voltage can be dynamically adjusted through various methods ! High single-

chip integration, simplified peripherals, low cost ! Built-in overvoltage protection module OVA, over-temperature protection module OTA

3. Applications

! Wireless Charger !

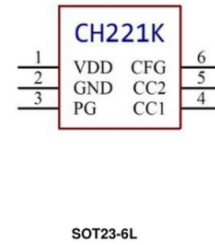
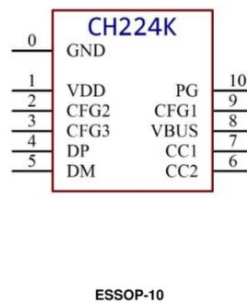
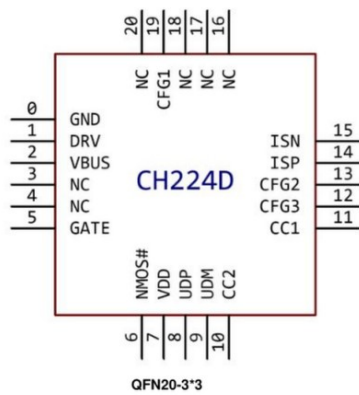
Laptop Charging Cable ! Lithium

Battery Small Appliances ! Lithium

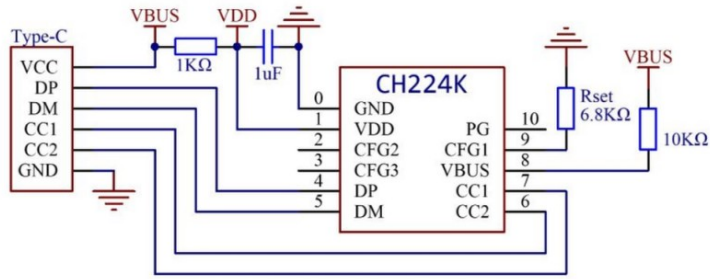
Battery Power Tool ! Power Bank

4. Pins

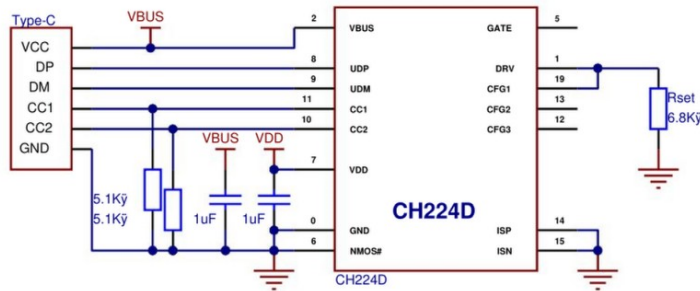
4.1. CH224 each package pin arrangement



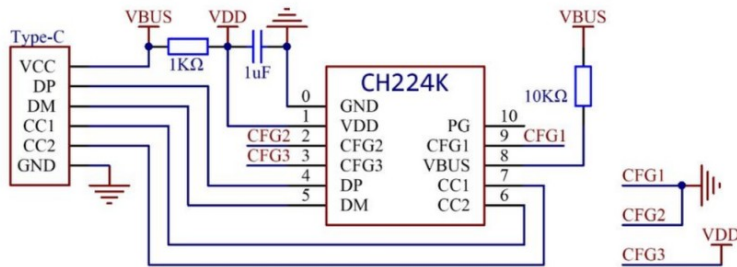
6.1. CH224K/CH224D use Type-C female port, single resistor configuration 9/12/15/20V (resistor configuration 6.8K Ω in the figure is 9v)



Rset resistance request voltage	
6.8K Ω	9V
24K Ω	12V
56K Ω	15V
NC	20V



6.2. CH224K uses Type-C female port, the level configuration is 5/9/12/15/20V (the level configuration in the figure is 12v)



CFG1	CFG2	CFG3	request voltage
1	1	1	5V
0	0	0	9V
0	0	1	12V
0	1	1	15V
0	1	0	20V